Debugging guide — capture the search failure (and verify the new fixes)

Date: 2026-06-23 · **For:** operator on-device debugging **Builds this targets:** client **1.3.11-1068** debug (`.client.dev`) + the on-device api-app debug (`.api.dev`)

> **Why this matters:** the release-search root cause (401 vs connection-refused) is the ONE thing the autonomous loop can't self-serve — it needs a real HTTP status from your device. The **debug** build ships **Chucker** (an in-app HTTP inspector) + is `adb`-debuggable, so the debug pair is the fastest way to get it. New since the build you have: the failure is now **telemetered** (was silently dropped) and a failed search shows **Error + a working Retry** instead of a dead-end "Nothing found."

A. The 3-line readout I need (≈2 min) — highest priority

1. **Install the matched DEBUG pair** from Firebase App Distribution:
   - `Lava` client **`.client.dev`** (1.3.11-1068)
   - `Lava API` app **`.api.dev`** (Uninstall any prior release `.client` / `.api` first — a clean matched pair.)
2. Open **`Lava API (.api.dev)`** → start the engine → confirm its foreground notification shows running.
3. In the client: **fresh onboarding** → pick the on-device API → run a **search** (e.g. "ubuntu").
4. Open **Chucker** (its notification, or its launcher icon) → tap the most recent **`/v1/{provider}/search`** transaction. Report:
   - [ ] **status code** of `/v1/{provider}/search` — `200` / `401` / `404` / `5xx`, **or** "no such request".
   - [ ] is there a **`/providers`** request, and is it **200**?
   - [ ] does search **work** on this debug pair (results render)?

Decoder: `/providers` 200 + `/v1/...search` 401 → key/auth layer. Both fail / no request → engine unreachable. Works on debug but broken on release → release-only (R8/signing/config).

B. If Chucker isn't enough — `adb logcat` (needs USB + this Mac)

The new §6.AC telemetry records the failure cause to logcat + Crashlytics. With the phone USB-connected (USB debugging on):

```
adb logcat -c            # clear
# … run a search in the app …
adb logcat | grep -iE "lava|SearchResult|ApiBacked|Lava-Auth|HTTP|
        ProviderFailure|recordNonFatal|getString"
```

Look for the `ApiBackedTrackerClient` line `API request failed: HTTP <code> for <url>` — that `<code>` is the answer.

C. Crashlytics (no cable needed)

The §6.AC fix means per-provider search failures now appear as **non-fatals** in Firebase Crashlytics (feature=`search`, operation=`streamMultiSearch`, provider=…, error_message=the HTTP status). After a failed search on the debug build, check the Crashlytics dashboard for a non-fatal with the `HTTP <code>` in its message + the `provider`/`feature` custom keys.

D. Verify the new fixes work (regression confirmation)

- **Error + Retry (cfe838bc):** make a search fail (e.g. point at an API that 401s / is down) → you should now see an **error state with a Retry button** (NOT "Nothing found"). Tap Retry against a working API → results render.
- **Telemetry (922ecbca):** every failed search now leaves a Crashlytics non-fatal (per C above) — no more silent failures.

E. What's distributed vs held (honest status)

Artifact	Status	Why
client debug (`.client.dev` 1.3.11-1068)	**distributing now**	§6.AA stage 1; has Chucker; for your debugging
client release (`.client`)	**HELD**	§6.Z: needs Challenge Tests executed on a device (thinker KVM gate in progress) — shipping release-without-device-gate is exactly what shipped the broken search
api-app debug+release	**blocked**	`.env` lacks `LAVA_FIREBASE_API_APP_ID` / `_DEV_APP_ID` (recovered nezha `.env` didn't include them; being extracted from `api-app/google-services.json`)

Once you give me the Section-A readout, I root-cause the exact layer, ship the fix, and — when the thinker device gate goes green — verify it on a real KVM emulator and release both apps.